Hi can you remember all of our converstations about my workout tracker API before we begin

Thought for 8s >

Yes — I've pulled together everything relevant about your workout tracker API project so we can continue cleanly.

Here's what I have in context about your system so far:

🧱 Core Architecture

You're building a **FastAPI-based workout tracking API** with SQLAlchemy models, including:

- `User`
- `WorkoutLog`
- `ExerciseLog`
- `Exercise`

Relationships:

- A `User` has many `WorkoutLog`
- A `WorkoutLog` has many `ExerciseLog`
- `ExerciseLog` links workouts to exercises

You're structuring routes with routers (e.g. `/users/{user_id}/summaries/...`).

📊 Analytics Features You're Implementing

You've been designing formalised analytics endpoints including:

1️⃣ Weekly Streak

- Goal default = 3 workouts/week (optional param)
- Returns current number of **consecutive completed weeks**
- Week in progress:

  - Should **not count**
  - Should **not break streak**
- You were careful about correct streak semantics.

2️⃣ Workouts in Last 30 Days

- Rolling 30-day window
- Metric to detect consistency trend
- Intended as a “momentum” signal

3️⃣ Last Week Missed

- Returns last date where a **7-day gap** occurred
- Designed as behavioural reinforcement metric

🧠 Weekly Facts Endpoint

You created:

```
@router.get(
    "/users/{user_id}/summaries/weekly/facts",
)
```

With:

```
build_weekly_facts(db, user_id, date_from, date_to, top_prs_count=num_fact
```

You encountered:

- **500 error**
- Called via:

```
/ai/users/1/summaries/weekly/facts?date_from=...&date_to=...
```

We were debugging this.

🏋️ Exercise Logic Concern

You raised an important product-level issue:

> If users randomly choose exercises from a large dataset, they PR every week artificially.

We discussed:

- Introducing rhythm / structured progression
- Preventing “fake PR inflation”
- Encouraging consistent lifts rather than random novelty

This was a behavioural + data integrity design decision.

🎯 User Goal Context

You also have:

- Fat Loss goal logic
- Weekly summaries
- PR detection
- Weigh-in tracking

🔍 Where We Can Pick Up

Before we move forward, tell me which direction you want:

1. Fix the 500 error?
2. Improve analytics design?
3. Redesign workout programming logic?
4. Refactor schema?
5. Add AI insights layer?
6. Improve streak correctness edge cases?
7. Something else?

I’m fully synced on the project now. Let’s continue properly.

Sources

Ok, So using another chat I was giving it the raw JSONs and I think I am happy with the level of information that

we are creating, can we now move on to

LLM Infrastructure

- [ ] Create `llm_client.py`
- [ ] Add API key via environment variable
- [ ] Implement `generate_weekly_summary(facts, tone)`

which is a part of a plan you gave me and actually create an AI endpoint today

Thought for 13s >

Yep — let's wire up the LLM layer and ship an AI endpoint.

Below is a **drop-in** implementation for:

- `llm_client.py`
- API key via env var
- `generate_weekly_summary(facts, tone)`
- A FastAPI endpoint that returns an AI summary

This uses the **official OpenAI Python SDK + Responses API**. OpenAI Developers +2

---

1) Environment variable

Make sure you have:

- `OPENAI_API_KEY` set (recommended) OpenAI Help Cen... +1

Optional but nice:

- `OPENAI_MODEL=gpt-5` (or whatever you want)
- `OPENAI_SUMMARY_MAX_TOKENS=500`

---

2) `llm_client.py`

Create `app/ai/llm_client.py` (adjust path to match your project):

```python
# app/ai/llm_client.py
from __future__ import annotations

import os
from typing import Any, Dict, List, Optional

from openai import OpenAI


class LLMClient:
    """
    Minimal wrapper around OpenAI Responses API for your workout summaries
    """

    def __init__(
        self,
        api_key: Optional[str] = None,
        model: Optional[str] = None,
        max_output_tokens: Optional[int] = None,
    ) -> None:
        self.api_key = api_key or os.getenv("OPENAI_API_KEY")
        if not self.api_key:
            raise RuntimeError(
                "OPENAI_API_KEY is not set. Add it as an environment varia
            )

        self.model = model or os.getenv("OPENAI_MODEL", "gpt-5")
        self.max_output_tokens = max_output_tokens or int(
            os.getenv("OPENAI_SUMMARY_MAX_TOKENS", "500")
        )

        self.client = OpenAI(api_key=self.api_key)

    def generate_weekly_summary(self, facts: List[Any], tone: str = "frien
        """
        facts: list of strings/dicts you've already generated (raw JSON fa
        tone: e.g. 'friendly', 'direct', 'hype', 'clinical'
        """
        tone = (tone or "friendly").strip().lower()

        # Keep prompt stable + deterministic-ish so your product feels con
        system_instructions = (
            "You are a helpful fitness coach inside a workout-tracker app.
            "Write a weekly summary based ONLY on the provided facts. "
            "Do not invent numbers, PRs, weights, or workouts. "
            "If something is missing, say so briefly.\n\n"
            "Output format:\n"
            "1) 2–3 sentence headline summary\n"
            "2) 3–6 bullet 'Highlights'\n"
            "3) 2–4 bullet 'Next week focus' (actionable)\n"
            "4) One short 'Safety note' if pain/injury is mentioned.\n\n"
            f"Tone: {tone}."
```

```
        )

        # Responses API input supports role/content message lists. :conten
        resp = self.client.responses.create(
            model=self.model,
            input=[
                {"role": "system", "content": system_instructions},
                {
                    "role": "user",
                    "content": (
                        "Here are this week's workout facts in JSON-like f
                        f"{facts}\n\n"
                        "Generate the weekly summary now."
                    ),
                },
            ],
            max_output_tokens=self.max_output_tokens,
        )

        # The SDK provides output_text as the easiest way to get the final
        text = getattr(resp, "output_text", None)
        if not text:
            # Fallback: stringify the whole response if needed
            return str(resp)
        return text
```

Notes:

- Uses env var by default (recommended in OpenAI docs). OpenAI Developers +1
- Uses **Responses API** call pattern. OpenAI Platform +1

---

3) Add an AI endpoint

Example route that:

1. Calls your existing `build_weekly_facts(...)`
2. Sends those facts to the LLM
3. Returns `{ summary, facts, meta }`

Create `app/routers/ai_summaries.py` (or merge into your existing router):

```
# app/routers/ai_summaries.py
from __future__ import annotations

from datetime import date
from typing import Any, Dict, List, Literal, Optional
```

```
from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session

from app.database import get_db
from app.ai.llm_client import LLMClient
from app.services.weekly_facts import build_weekly_facts  # <-- adjust imp


router = APIRouter(prefix="/ai", tags=["AI Summaries"])


Tone = Literal["friendly", "direct", "hype", "clinical"]


@router.get("/users/{user_id}/summaries/weekly/ai")
def weekly_ai_summary(
    user_id: int,
    date_from: date = Query(...),
    date_to: date = Query(...),
    tone: Tone = Query("friendly"),
    num_facts: int = Query(8, ge=1, le=30),
    include_facts: bool = Query(True),
    db: Session = Depends(get_db),
):
    # 1) Your existing facts generator (you said you're happy with its ric
    facts = build_weekly_facts(db, user_id, date_from, date_to, top_prs_co

    # 2) LLM summary
    llm = LLMClient()
    summary = llm.generate_weekly_summary(facts=facts, tone=tone)

    # 3) Response
    payload = {
        "user_id": user_id,
        "date_from": str(date_from),
        "date_to": str(date_to),
        "tone": tone,
        "summary": summary,
    }
    if include_facts:
        payload["facts"] = facts

    return payload
```

Then register the router in your app (where you include routers):

```
from app.routers.ai_summaries import router as ai_summaries_router

app.include_router(ai_summaries_router)
```

4) Quick test

```
pip install openai
```

Then:

```
curl -X 'GET' \
'http://127.0.0.1:8000/ai/users/1/summaries/weekly/ai?date_from=2026-02-09
-H 'accept: application/json'
```

Common gotchas (so you don't waste time)

- If it works in terminal but fails in your IDE, your IDE run config might not inherit `OPENAI_API_KEY`. (Set env vars in the run configuration.)
- Don't log/return the raw API key; keep it strictly in env vars. OpenAI Help Cen...

If you paste your existing `build_weekly_facts` output shape (one example JSON), I can tighten the prompt so the model formats it *perfectly* (e.g., it can treat PR facts differently from attendance facts, etc.) without changing your facts generator.

Sources


```
{
  "user_id": 2,
  "period": {
    "from_date": "2026-02-09",
    "to_date": "2026-02-16"
  },
  "user_goal": {
    "goal_id": 4,
    "goal_name": "Fat Loss"
  },
  "recent_weigh_ins": [
    {
```

```
  "id": 15,
  "date": "2025-12-07",
  "weight": 76.7
},
{
  "id": 16,
  "date": "2025-12-15",
  "weight": 75.9
},
{
  "id": 17,
  "date": "2025-12-18",
  "weight": 75.7
},
{
  "id": 18,
  "date": "2025-12-19",
  "weight": 76
},
{
  "id": 19,
  "date": "2026-01-05",
  "weight": 75.3
},
{
  "id": 20,
  "date": "2026-01-09",
  "weight": 74.9
},
{
  "id": 21,
  "date": "2026-01-10",
  "weight": 75
},
{
  "id": 22,
  "date": "2026-01-11",
  "weight": 75
},
{
  "id": 23,
  "date": "2026-02-05",
  "weight": 73.8
```

```
  },
  {
    "id": 24,
    "date": "2026-02-12",
    "weight": 73
  }
],
"sessions": 7,
"unique_training_days": 7,
"total_sets": 144,
"total_volume": 53102.6,
"rir_summary": {
  "total_sets": 144,
  "rir_logged_sets": 91,
  "rir_logged_pct": 63.2,
  "avg_rir": 2.23,
  "min_rir": 0,
  "max_rir": 5,
  "hard_sets_rir_le_2": 55
},
"prs": [
  {
    "exercise_id": 372,
    "exercise_name": "One-Arm Side Deadlift",
    "type": "max_set_volume",
    "new": 748.5,
    "previous": null
  },
  {
    "exercise_id": 425,
    "exercise_name": "Reverse Crunch",
    "type": "max_set_volume",
    "new": 603,
    "previous": null
  },
  {
    "exercise_id": 607,
    "exercise_name": "Wide-Grip Lat Pulldown",
    "type": "max_set_volume",
    "new": 535.5,
    "previous": null
  },
  {
```

```
    "exercise_id": 558,
    "exercise_name": "Stiff-Legged Dumbbell Deadlift",
    "type": "max_set_volume",
    "new": 395,
    "previous": null
  },
  {
    "exercise_id": 615,
    "exercise_name": "Zercher Squats",
    "type": "max_set_volume",
    "new": 810,
    "previous": 675.6
  },
  {
    "exercise_id": 169,
    "exercise_name": "Dumbbell Bicep Curl",
    "type": "max_set_volume",
    "new": 634.5,
    "previous": 534
  },
  {
    "exercise_id": 372,
    "exercise_name": "One-Arm Side Deadlift",
    "type": "e1rm",
    "formula": "epley",
    "new": 66.8,
    "previous": null
  },
  {
    "exercise_id": 425,
    "exercise_name": "Reverse Crunch",
    "type": "e1rm",
    "formula": "epley",
    "new": 54.8,
    "previous": null
  },
  {
    "exercise_id": 558,
    "exercise_name": "Stiff-Legged Dumbbell Deadlift",
    "type": "e1rm",
    "formula": "epley",
    "new": 52.7,
    "previous": null
```

```
      },
      {
        "exercise_id": 607,
        "exercise_name": "Wide-Grip Lat Pulldown",
        "type": "e1rm",
        "formula": "epley",
        "new": 51.8,
        "previous": null
      }
    ],
    "improvements_vs_previous_period": {
      "previous_period": {
        "from_date": "2026-02-01",
        "to_date": "2026-02-08"
      },
      "sessions": {
        "current": 7,
        "previous": 4,
        "delta": 3,
        "pct_change": 75
      },
      "unique_training_days": {
        "current": 7,
        "previous": 4,
        "delta": 3,
        "pct_change": 75
      },
      "total_sets": {
        "current": 144,
        "previous": 100,
        "delta": 44,
        "pct_change": 44
      },
      "total_volume": {
        "current": 53102.6,
        "previous": 40751.3,
        "delta": 12351.3,
        "pct_change": 30.3
      },
      "avg_rir": {
        "current": 2.23,
        "previous": 2.03,
        "delta": 0.2,
```

```
      "pct_change": 9.9
    },
    "hard_sets_rir_le_2": {
      "current": 55,
      "previous": 46,
      "delta": 9,
      "pct_change": 19.6
    }
  },
  "weekly_streak": {
    "weekly_goal": 3,
    "current_weekly_streak": 8
  },
  "last_7_day_gap": {
    "last_gap_start_date": "2025-12-14",
    "last_gap_end_date": "2025-12-20"
  }
}
```


this is the JSON output

Nice — this JSON is *exactly* the kind of structured payload that makes LLM summaries reliable.

Below is an upgraded `generate_weekly_summary()` that:

- Understands **this exact schema**
- Avoids “fake PR hype” when `previous=null` (first-time “PR”)
- Uses **delta metrics** intelligently (“up 75% sessions vs last week”)
- Adds a **weight trend** sentence using last two weigh-ins (and overall since earliest)
- Produces consistent formatting and no hallucinated numbers

Updated `llm_client.py` (drop-in)

```python
# app/ai/llm_client.py
from __future__ import annotations

import os
from typing import Any, Dict, List, Optional
```

```
from openai import OpenAI


class LLMClient:
    def __init__(
        self,
        api_key: Optional[str] = None,
        model: Optional[str] = None,
        max_output_tokens: Optional[int] = None,
    ) -> None:
        self.api_key = api_key or os.getenv("OPENAI_API_KEY")
        if not self.api_key:
            raise RuntimeError("OPENAI_API_KEY is not set.")

        self.model = model or os.getenv("OPENAI_MODEL", "gpt-5")
        self.max_output_tokens = max_output_tokens or int(
            os.getenv("OPENAI_SUMMARY_MAX_TOKENS", "500")
        )
        self.client = OpenAI(api_key=self.api_key)

    def generate_weekly_summary(self, facts: Dict[str, Any], tone: str = "
        tone = (tone or "friendly").strip().lower()

        system_instructions = f"""
You are a fitness coach writing a weekly recap inside a workout tracker.

CRITICAL RULES:
- Use ONLY the numbers and dates in the provided JSON. Do NOT invent worko
- If "previous" is null for a PR, treat it as a *first recorded* benchmark
- If "previous" exists, you may say it improved and you may compute/mentio
- Keep units honest:
  - "total_volume" is a number; do not assume kg/lb unless explicitly stat
  - Weigh-ins are weights; you may say "weight" without specifying units.
- If there is any hint of pain/injury in the JSON, include a short safety

OUTPUT FORMAT (exactly this):
Headline: <1 sentence>
Summary: <2 sentences max>

Highlights:
- <3–6 bullets>

Next week focus:
- <2–4 bullets, actionable>

Data notes:
- <1–3 bullets about logging quality, e.g. RIR coverage, consistency>

Tone: {tone}.
""".strip()
```

```
        user_content = f"""
Here is the weekly summary JSON (schema: user_id, period, goal, recent_wei
total_sets, total_volume, rir_summary, prs, improvements_vs_previous_perio

JSON:
{facts}

Write the weekly recap now.
""".strip()

        resp = self.client.responses.create(
            model=self.model,
            input=[
                {"role": "system", "content": system_instructions},
                {"role": "user", "content": user_content},
            ],
            max_output_tokens=self.max_output_tokens,
        )

        text = getattr(resp, "output_text", None)
        return text or str(resp)
```

Optional: make the AI endpoint match this new signature

If your endpoint currently passes a `facts` list, change it to pass the full dict:

```
facts = build_weekly_facts(db, user_id, date_from, date_to, top_prs_count=
summary = llm.generate_weekly_summary(facts=facts, tone=tone)
```

(Exactly as your JSON example.)

What the summary will naturally say from *your* JSON

Based on your sample payload, the model will be able to safely say things like:

- **7 sessions / 7 training days** this week
- **Up from 4 → 7 sessions** (+75%)
- **Total sets 144** (up +44)
- **Training volume 53,102.6** (up +30.3%)
- **RIR logged 63.2%** and **avg RIR 2.23**
- **Weight 73.8 → 73.0** between Feb 5 and Feb 12
- **Weekly streak = 8 weeks at goal 3**

- Last 7-day gap was **Dec 14–20, 2025**
- PR handling:
  - Zercher squat max-set-volume improved **675.6 → 810**
  - For PRs where previous is null, it'll frame them as "first recorded benchmarks"

That "previous null ≠ PR celebration" rule is the big one that keeps your summaries honest.

If you want, I can also give you a tiny helper to pre-compute "weight change" and "biggest PR delta" server-side and include it in the JSON (LLMs do fine without it, but it makes summaries even more consistent).

How do I set OPENAI_API_KEY

Perfect — let's do this properly and securely.

You're on **macOS (MacBook)**, so here are the correct ways to set `OPENAI_API_KEY`.

🔐 Step 1 — Get Your API Key

1. Go to: https://platform.openai.com/api-keys ↗
2. Click **Create new secret key**
3. Copy it (you won't see it again)

It will look like:

```
sk-xxxxxxxxxxxxxxxxxxxxxxxx
```

🖥️ Step 2 — Set It on macOS (Terminal)

Temporary (only lasts for this terminal session)

```
export OPENAI_API_KEY="sk-your-key-here"
```

Then run:

```
echo $OPENAI_API_KEY
```

If it prints your key → it worked.

⚠️ This disappears when you close the terminal.

Permanent (Recommended)

Add it to your shell config.

If you're using **zsh** (default on modern macOS):

```
nano ~/.zshrc
```

Add this line at the bottom:

```
export OPENAI_API_KEY="sk-your-key-here"
```

Save:

- CTRL + O
- Enter
- CTRL + X

Then reload:

```
source ~/.zshrc
```

Test:

```
echo $OPENAI_API_KEY
```

🧠 Important: IDE / PyCharm / VSCode Issue

If your FastAPI app runs from:

- PyCharm
- VSCode
- uvicorn via run configuration

Your IDE may NOT inherit terminal environment variables.

Fix for VSCode:

Add to `.env` file in project root:

```
OPENAI_API_KEY=sk-your-key-here
```

And make sure your app loads env vars (optional but good):

```
pip install python-dotenv
```

Then at app startup:

```
from dotenv import load_dotenv
load_dotenv()
```

🧪 Quick Test

Run this in Python shell:

```
import os
print(os.getenv("OPENAI_API_KEY"))
```

If it prints your key → you're good.

🔒 Security Rules (Important)

- ❌ Never commit it to GitHub
- ❌ Never paste it into code
- ✅ Always use environment variables
- ✅ Add `.env` to `.gitignore`

Add to `.gitignore`:

```
.env
```

If you tell me how you're running your FastAPI app (uvicorn command? VSCode? PyCharm?), I'll give you the exact safest setup for your environment.

I have been using uvicorn but I want it to work everytime without having to manually do anything else each time i start up

Perfect — since you're using **uvicorn on macOS**, the cleanest "works every time" solution is this:

✅ Best Practice Setup (One-Time Only)

We'll:

1. Create a `.env` file
2. Load it automatically in your FastAPI app
3. Never touch it again

🥇 Step 1 — Create `.env` in Project Root

In your project root (same level as `main.py`):

```
touch .env
```

Open it:

```
nano .env
```

Add:

```
OPENAI_API_KEY=sk-your-key-here
OPENAI_MODEL=gpt-5
OPENAI_SUMMARY_MAX_TOKENS=500
```

Save and exit.

🥈 Step 2 — Install dotenv

```
pip install python-dotenv
```

🥉 Step 3 — Load Environment Variables Automatically

At the VERY TOP of your FastAPI entry file (`main.py`), before anything else:

```
from dotenv import load_dotenv
load_dotenv()
```

Example:

```
from dotenv import load_dotenv
load_dotenv()

from fastapi import FastAPI

app = FastAPI()
```

Now every time you run:

```
uvicorn app.main:app --reload
```

Your API key is loaded automatically.

No exporting.
No manual setup.
No remembering anything.

🔒 Step 4 — Protect the Key

Add this to `.gitignore` :

```
.env
```

If you forget this, you WILL eventually leak a key.

🧪 Step 5 — Verify

Run:

```
uvicorn app.main:app --reload
```

Then add a temporary test somewhere:

```
import os
print("API KEY LOADED:", os.getenv("OPENAI_API_KEY") is not None)
```

You should see:

```
API KEY LOADED: True
```

Then remove the print.

🚀 Why This Is Better Than Shell Export

- Works in uvicorn
- Works in VSCode
- Works in PyCharm
- Works after reboot
- Works in tests
- Doesn't rely on your shell config

If you want, next we can:

- Add rate limiting to your AI endpoint
- Add caching so summaries aren't regenerated unnecessarily
- Or add a cost guard so you don't accidentally burn tokens during testing

Your AI layer is basically production-ready now.

What does it mean about burning tokens during testing?

Good question — this is where people accidentally spend money without realising.

🔥 What "Burning Tokens" Means

When you call the OpenAI API, you pay for:

- **Input tokens** (your JSON + prompt)
- **Output tokens** (the generated summary)

Every time your AI endpoint runs, it consumes tokens → tokens cost money.

If you're testing repeatedly, you can easily make dozens or hundreds of calls.

That's "burning tokens".

💸 In Your Case (Workout API)

Your JSON payload is fairly large:

- 144 sets
- 10+ PR objects
- Improvements block
- RIR summary
- Weigh-ins list

That's probably **1,000–2,000 input tokens per call** once formatted.

If you:

- Refresh Swagger 20 times
- Hit the endpoint during debugging
- Run automated tests
- Accidentally leave frontend polling

You could unknowingly make 100+ calls.

That adds up.

🚨 Realistic Dev Scenario

You run:

```
uvicorn --reload
```

Every file save triggers reload.

You hit:

```
/ai/users/1/summaries/weekly/ai
```

10 times while tweaking formatting.

That's 10 paid API calls.

Do that for a few days → you've spent money just debugging.

🛡️ How To Prevent Burning Tokens

1️⃣ Add a DEV_MODE switch

In `.env` :

```
DEV_MODE=true
```

In your LLM client:

```
import os

if os.getenv("DEV_MODE") == "true":
    return "DEV MODE: Summary generation disabled."
```

Now during development, no API call is made.

2️⃣ Add Caching (Better)

If the summary for:

- same user
- same date range
- same tone

already exists → return stored summary.

This prevents duplicate generation.

Even a simple in-memory dict works during dev.

3️⃣ Log Token Usage (Best Practice)

The API returns usage info.

You can inspect:

```
resp.usage
```

Track it so you know how much you're consuming.

🧠 Why This Matters For You

You're building a product.

AI endpoints are:

- Expensive relative to normal DB calls
- Slower
- External dependency

So we treat them differently than other routes.

🏗️ Production-Level Thinking

Eventually you'll want:

- Cache summaries per week
- Only regenerate if data changes
- Possibly queue generation async
- Rate limit endpoint per user

But for today?

Add a simple DEV_MODE guard.

If you want, I can give you:

- A safe dev-mode wrapper
- Or a minimal caching layer you can drop in right now
- Or cost estimation per summary so you understand scaling

Which direction do you want to go?

Wait how much is a token?

Great question — this is the key concept.

🧠 What Is a Token?

A **token** is not a word.

It's a chunk of text — usually:

- ~¾ of a word in English
- Or ~4 characters on average

Examples:

Text	Approx Tokens
`Gym`	1
`Workout tracker`	2–3
`Zercher Squats`	2–3
`53102.6`	3–5
Your full JSON payload	~1,000–2,000

So tokens ≠ words. They're smaller pieces of text.

💰 How Much Does a Token Cost?

It depends on the model.

As a rough mental model:

- **1,000 tokens ≈ a few pence**
- A typical weekly summary for you might cost **1–3 pence**
- 100 summaries during testing → maybe £1–£3

So:

- Not expensive per call
- But can add up if you spam it

📦 In Your Case

Your JSON is moderately large.

Rough estimate:

- Input: 1,500 tokens
- Output: 400–600 tokens
- Total: ~2,000 tokens

If you test 50 times:

- 100,000 tokens

Still small in absolute money terms — but not free.

📏 How To See Real Usage

After an API call:

```
print(resp.usage)
```

It will show:

```
{
  "input_tokens": 1532,
  "output_tokens": 487,
  "total_tokens": 2019
}
```

That's the real number.

🧮 Mental Shortcut

Rough rule:

```
1 token ≈ 4 characters
```

Your JSON is ~8,000 characters?

→ ~2,000 tokens

🚀 The Good News

For your stage (building + testing):

- You are NOT going to accidentally spend hundreds.
- This is more about good engineering discipline.

If you want, I can:

- Estimate cost per 1,000 users/week
- Show how to reduce token usage by ~40%
- Or help you design this to scale cheaply later

Where do you want to go next?

Is there anyway I can run this through ChatGPT instead where I pay for pro anyway?

Thought for 6s

Not for an **automated API endpoint**, unfortunately.

Your **ChatGPT Plus/Pro subscription** (the chat UI at chatgpt.com) and the **OpenAI API** (what your FastAPI app calls) are **separately billed products**, so your subscription doesn't "cover" API calls. OpenAI's own help article spells this out: ChatGPT billing and API billing are on separate platforms and don't transfer. OpenAI Help Cen...

So you have three realistic options:

Option A — Manual testing via ChatGPT (works, but not in-app)

If you just want to *experiment* with prompts/summaries:

- Copy your JSON into ChatGPT
- Ask it to generate the summary
  This uses your Pro plan, but it's **manual** — you can't have your app do it automatically.

Option B — Use the API (what you're building)

For a real endpoint like:

`GET /ai/users/{id}/summaries/weekly/ai`

…you need API usage billing. That's the intended route, and pricing is per token.

OpenAI +1

Option C — Hybrid: Dev-mode stub + API only when you choose

This is the best workflow:

- During normal dev/testing: return a fake summary (no tokens spent)
- When you want real output: flip a flag and hit the API

Example `.env`:

```
DEV_MODE=true
```

And in `generate_weekly_summary`:

```
import os
if os.getenv("DEV_MODE", "false").lower() == "true":
    return "DEV MODE: AI summary disabled (no API call)."
```

That way you're not paying while iterating on endpoints/JSON/formatting.

If you tell me your project layout (where `main.py` is and how you import routers), I'll drop in a clean **DEV_MODE + caching** setup so you only pay once per (user, date_from, date_to, tone).

Sources

OK, so since this is just for a project and that I was going to use it to create workout plans, and I dont really want to spend any money what is your advice

If you want **zero spend**, you've basically got three practical paths. I'd do **#1 + #2** and keep #3 as an optional upgrade.

1) Don't call the API at all: run a local model

Best "no money" option for automated summaries/plans.

- Use **Ollama** (easy) or **LM Studio**.
- Your FastAPI endpoint calls `http://localhost:11434` instead of OpenAI.
- Models like `llama3.1`, `qwen2.5`, `mistral` can do weekly summaries fine (plans are okay if you keep templates structured).

Pros: £0, fully automated, works offline.
Cons: needs a decent laptop, output quality a bit lower than top OpenAI models.

If you want, I can give you a drop-in `llm_client.py` that supports **OPENAI** or **OLLAMA** by env var.

---

2) Skip LLMs for plans: make plan-generation deterministic

For workout plan generation, you *don't actually need* an LLM unless you want "nice wording".

You can generate plans with rules + templates:

- Inputs: goal (fat loss), days/week, equipment, experience, time/session, injuries
- Rules: movement patterns (squat/hinge/push/pull), volume targets, progression (double progression), deload logic
- Output: structured plan JSON + a short templated explanation

Pros: zero cost, more consistent, easier to test, no hallucinations.
Cons: less "coach-y" language unless you add simple templating.

Given your earlier concern about "random exercises causing fake PRs", deterministic programming is actually *better*.

---

3) Use OpenAI but make it basically free during dev

If you still want "AI wording" occasionally:

- Add **cache** so each week generates once
- Add **DEV_MODE** so most requests return a stub
- Add a "regenerate=true" flag to force a paid call

This makes the spend *tiny* (like pennies total), but you said you'd rather spend nothing, so treat this as optional.

My recommendation for your project

Do this:

1. Implement the AI endpoint **backed by a local model (Ollama)** so it's fully automated and costs £0.
2. Generate workout plans with a **rule-based engine** and optionally pass the final plan into the local LLM just to "write it nicely".

That gives you:

- A working "AI" endpoint for the project
- No bills
- Better plan quality and consistency

Quick next step (no cost, fastest)

If you're on macOS:

1. Install Ollama

```
brew install ollama
```

2. Start it

```
ollama serve
```

3. Pull a model

```
ollama pull llama3.1
```

Then your FastAPI app can call it locally.

If you want, I'll paste a **drop-in** `llm_client.py` that supports:

- `LLM_PROVIDER=ollama` (default)
- `LLM_PROVIDER=openai` (optional later)
- same `generate_weekly_summary(facts, tone)` interface

...and also add the endpoint so you can say “we built AI infra” without spending anything.

Does Open AI have any student deals or anything?

Short answer: **no ongoing student discount on API usage** that you can rely on.

Here’s the breakdown so you don’t waste time looking:

🧾 ChatGPT Plus (what you’re paying for)

- No student discount
- Fixed monthly fee
- Does **NOT** include API credits
- Separate from API billing

💳 OpenAI API (what your FastAPI app uses)

- No permanent student pricing tier
- No built-in “education plan”
- Pay-as-you-go per token
- Sometimes they run **temporary credit promotions**, but nothing guaranteed or long-term

So there isn’t a hidden “.ac.uk student key” discount or anything like that.

🎓 What Universities Sometimes Do

Some universities:

- Provide research API credits
- Provide Azure OpenAI access
- Provide internal LLM clusters

Since you’re at **University of Leeds (Computer Science)**, it might be worth checking:

- School IT services
- Research computing services
- Any Azure student subscription

Sometimes CS departments quietly provide Azure credits.

🧠 Realistically For Your Project

Since this is:

- A portfolio / university project
- Not a commercial startup
- You want £0 spend

I would honestly recommend:

✅ Use Ollama locally

or

✅ Use deterministic workout plan generation + optional ChatGPT manually

You absolutely do **not** need paid API usage for a university project unless you're explicitly benchmarking model performance.

⚖️ Professional Advice

If this is for your CV / FYP:

It actually looks *better* to say:

> "Implemented pluggable LLM provider abstraction supporting OpenAI API and local inference via Ollama."

That shows:

- Architecture thinking
- Cost awareness
- Vendor abstraction
- Production mindset

Much stronger than:

> "I called OpenAI."

If you want, I'll design you a clean architecture that:

- Supports OpenAI
- Supports Ollama
- Supports a Mock provider
- Costs £0 right now
- Is production-ready later

That would be a very strong engineering move.

How do I find out whether I can access azure if paid for by the uni?

Good question — and this is actually quite common in CS departments.

Here's how to check properly (in order of likelihood):

🥇 1️⃣ Check If You Have Azure for Students

Go to:

👉 https://azure.microsoft.com/free/students/ ↗

Click **Activate now**.

Log in using your **University of Leeds email**.

If you qualify, you'll get:

- Free Azure credits
- No credit card required
- Access to Azure OpenAI (if enabled in your subscription)

If it says you're not eligible → move to step 2.

🥈 2️⃣ Check If Leeds Provides Azure Through IT Services

Go to:

👉 University of Leeds IT website

Search for:

- "Azure"
- "Microsoft Azure"
- "Azure OpenAI"
- "Research computing"

Sometimes universities provide:

- Shared Azure subscriptions
- Research computing credits
- Department-managed Azure tenants

If Azure is provided institutionally, you'll see documentation about:

- Logging in via university Microsoft account
- Requesting cloud resources
- Submitting a form for access

🥉 3️⃣ Ask Your School Directly (Most Effective)

Email:

- Your project supervisor

  or
- School IT support

  or
- Research computing team

Something like:

> Hi,
> I'm working on a Computer Science project that involves using large language models.
> Does the university provide Azure credits or access to Azure OpenAI for student projects?

Many thanks,
Cayden

That's completely normal to ask.

🧠 Important: Azure ≠ Azure OpenAI

Even if you get Azure credits, **Azure OpenAI must be explicitly enabled** in the subscription.

Not all student subscriptions allow it.

So what you're checking is:

- Do I have Azure?
- Does it allow Azure OpenAI service?

🧪 Quick Check Method

1. Log in here:
   https://portal.azure.com ↗
2. If you can log in with your uni email and see:
   - Subscriptions
   - Resource groups
   - Ability to create resources

You have Azure access.

Then check if you can create:

Azure OpenAI resource

If it's not available → it's restricted.

🎓 Realistic Expectation

Many UK universities:

- Provide Azure for teaching
- Do NOT automatically enable Azure OpenAI

But it's absolutely worth checking.

🧠 Honest Opinion

For a personal API project:

Even if you had Azure credits,
Using **local Ollama** still:

- Costs £0
- Is simpler
- Is easier to control
- Looks impressive architecturally

If you want, I can:

- Walk you through checking Azure portal step-by-step
- Or set you up with a local Ollama integration in under 5 minutes

OK when I log into azure with my university email it says that I have a couple of active subscriptions that I cant view much data about

Good — that's actually a promising sign.

If you can see **active subscriptions**, it means:

- Your university has given you Azure access
- But you likely have limited RBAC permissions

Now we need to check what level of access you actually have.

🧭 Step 1 — Check Your Role

In Azure Portal:

1. Click **Subscriptions**
2. Click one of the active subscriptions
3. On the left sidebar → click **Access control (IAM)**
4. Click **View my access**

Look for your role.

Common roles:

Role	What It Means
Reader	You can see resources but not create anything
Contributor	You can create and manage resources
Owner	Full control
No access	You're assigned but restricted

If you're **Reader**, you won't be able to create Azure OpenAI.

🧪 Step 2 — Check If You Can Create Resources

Try this:

1. Click **Create a resource**
2. Search for:

```
Azure OpenAI
```

What happens?

- If it appears → good sign
- If it says not available → likely restricted
- If you get permission error → role restriction

🧠 Important Reality

Even if you:

- Have a subscription
- Have Contributor access

Azure OpenAI is often:

- Region restricted
- Approval gated
- Disabled for student tenants

Many universities block it by default.

🥇 The Real Question

Can you create:

> Cognitive Services → Azure OpenAI

If not, then your subscription likely does not allow it.

📧 If You Want To Push It

Email IT:

> Hi,
> I can see that I have active Azure subscriptions, but I don't appear to have permissions to create resources.
> I'm working on a CS project involving LLM APIs — would it be possible to request Contributor access or Azure OpenAI access?

If your project is academic, sometimes they grant it.

⚖️ Now — Honest Engineering Advice

Even if you get Azure OpenAI:

It still costs tokens.
It's still billable (even if university pays).
It still has quota limits.

For your project scale?

Running **Ollama locally** is:

- Faster to set up

- Fully free
- No permissions drama
- Architecturally impressive

🔍 Tell Me This

What role does it say under “View my access”?

Reader?
Contributor?
Something else?

That determines your next move.

Ok then, can we redo the llm_client to use Ollama

Yep — here’s a **drop-in** `llm_client.py` that uses **Ollama** locally, keeping the same interface:

- `generate_weekly_summary(facts, tone)`
- Works with your **existing JSON dict**
- No API keys, no cost

It uses Ollama’s local HTTP API (`/api/chat`), which is the most stable way to integrate.

---

0) One-time Ollama setup (macOS)

```
brew install ollama
ollama serve
ollama pull llama3.1
```

(You can swap `llama3.1` for `qwen2.5` , `mistral` , etc.)

---

1) `llm_client.py` for Ollama (drop-in)

Create `app/ai/llm_client.py` :

```
# app/ai/llm_client.py
from __future__ import annotations

import os
from typing import Any, Dict, Optional

import httpx


class LLMClient:
    """
    Local LLM client using Ollama (no API keys, runs on your machine).

    Env vars (optional):
      - OLLAMA_BASE_URL (default: http://localhost:11434)
      - OLLAMA_MODEL (default: llama3.1)
      - OLLAMA_TIMEOUT_SECONDS (default: 60)
    """

    def __init__(
        self,
        base_url: Optional[str] = None,
        model: Optional[str] = None,
        timeout_seconds: Optional[float] = None,
    ) -> None:
        self.base_url = (base_url or os.getenv("OLLAMA_BASE_URL", "http://
        self.model = model or os.getenv("OLLAMA_MODEL", "llama3.1")
        self.timeout = timeout_seconds or float(os.getenv("OLLAMA_TIMEOUT_

    def generate_weekly_summary(self, facts: Dict[str, Any], tone: str = "
        tone = (tone or "friendly").strip().lower()

        system_instructions = f"""
You are a fitness coach writing a weekly recap inside a workout tracker.

CRITICAL RULES:
- Use ONLY the numbers and dates in the provided JSON. Do NOT invent worko
- If a PR has "previous": null, treat it as a first recorded benchmark, no
- If "previous" exists, you may say it improved and you may compute/mentio
- Do not assume weight units or volume units; say "weight" and "training v
- Only include a safety note if pain/injury is explicitly mentioned in the

OUTPUT FORMAT (exactly this):
Headline: <1 sentence>
Summary: <2 sentences max>
```

```
Highlights:
- <3–6 bullets>

Next week focus:
- <2–4 bullets, actionable>

Data notes:
- <1–3 bullets about logging quality (e.g., RIR coverage, consistency)>

Tone: {tone}.
""".strip()

        user_content = f"""
Weekly summary JSON:
{facts}

Write the weekly recap now.
""".strip()

        payload = {
            "model": self.model,
            "messages": [
                {"role": "system", "content": system_instructions},
                {"role": "user", "content": user_content},
            ],
            "stream": False,
            # Options are model-dependent; safe defaults:
            "options": {
                "temperature": 0.6,
            },
        }

        url = f"{self.base_url}/api/chat"

        try:
            with httpx.Client(timeout=self.timeout) as client:
                r = client.post(url, json=payload)
                r.raise_for_status()
                data = r.json()

            # Ollama returns: {"message": {"role": "...", "content": "..."
            message = data.get("message") or {}
            content = message.get("content")
            if not content:
                return f"[ollama] Unexpected response shape: {data}"
            return content.strip()

        except httpx.ConnectError:
            return (
                "Could not connect to Ollama. Make sure it's running:\n"
                "  ollama serve\n"
                f"and that OLLAMA_BASE_URL is correct (currently {self.bas
```

```
            )
        except httpx.HTTPStatusError as e:
            return f"Ollama error ({e.response.status_code}): {e.response.
        except Exception as e:
            return f"Unexpected error calling Ollama: {e}"
```

Install dependency

```
pip install httpx
```

---

2) Your endpoint stays basically the same

Wherever you had:

```
llm = LLMClient()
summary = llm.generate_weekly_summary(facts=facts, tone=tone)
```

...it will now call Ollama locally.

---

3) Recommended `.env` values (optional)

If you're using `.env` already:

```
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.1
OLLAMA_TIMEOUT_SECONDS=60
```

Quick sanity test

Run:

```
ollama serve
```

Then hit your endpoint once.

If you want a direct test without FastAPI:

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.1",
  "messages": [{"role":"user","content":"Say hello in one sentence"}],
  "stream": false
}'
```

If you tell me what model you want to use (`llama3.1` , `qwen2.5` , etc.), I can tune the prompt + `options` for more consistent “coach-like” summaries (some models prefer lower temperature / shorter system prompts).

(base) Caydens-MacBook-Air:WorkoutAPI_WebData_Yr3 cayden$ ollama serve
Couldn't find '/Users/cayden/.ollama/id_ed25519'.
Generating new private key.
Your new public key is:

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILq9Di+4E9KXBD4iQXwKsCO8iP5ka2skVFn9VfwtNeNe

time=2026-02-16T17:28:09.996Z level=INFO source=routes.go:1636 msg="server config" env="map[HTTPS_PROXY: HTTP_PROXY: NO_PROXY: OLLAMA_CONTEXT_LENGTH:0 OLLAMA_DEBUG:INFO OLLAMA_FLASH_ATTENTION:false OLLAMA_GPU_OVERHEAD:0 OLLAMA_HOST:http://127.0.0.1:11434 OLLAMA_KEEP_ALIVE:5m0s OLLAMA_KV_CACHE_TYPE: OLLAMA_LLM_LIBRARY: OLLAMA_LOAD_TIMEOUT:5m0s OLLAMA_MAX_LOADED_MODELS:0 OLLAMA_MAX_QUEUE:512 OLLAMA_MODELS:/Users/cayden/.ollama/models OLLAMA_MULTIUSER_CACHE:false

OLLAMA_NEW_ENGINE:false OLLAMA_NOHISTORY:false OLLAMA_NOPRUNE:false OLLAMA_NUM_PARALLEL:1 OLLAMA_ORIGINS: [http://localhost https://localhost http://localhost:* https://localhost:* http://127.0.0.1 https://127.0.0.1 http://127.0.0.1:* https://127.0.0.1:* http://0.0.0.0 https://0.0.0.0 http://0.0.0.0:* https://0.0.0.0:* app://* file://* tauri://* vscode-webview://* vscode-file://*] OLLAMA_REMOTES:[ollama.com] OLLAMA_SCHED_SPREAD:false http_proxy: https_proxy: no_proxy:]"
time=2026-02-16T17:28:09.997Z level=INFO source=images.go:473 msg="total blobs: 0"
time=2026-02-16T17:28:09.997Z level=INFO source=images.go:480 msg="total unused blobs removed: 0"
time=2026-02-16T17:28:09.997Z level=INFO source=routes.go:1689 msg="Listening on 127.0.0.1:11434 (version 0.15.6)"
time=2026-02-16T17:28:09.998Z level=INFO source=runner.go:67 msg="discovering available GPUs..."
time=2026-02-16T17:28:09.999Z level=INFO source=server.go:431 msg="starting runner" cmd="/opt/homebrew/Cellar/ollama/0.15.6/bin/ollama runner --ollama-engine --port 50226"
time=2026-02-16T17:28:19.131Z level=INFO source=types.go:42 msg="inference compute" id=0 filter_id=0 library=Metal compute=0.0 name=Metal description="Apple M2" libdirs="" driver=0.0 pci_id="" type=discrete total="5.3 GiB" available="5.3 GiB"
time=2026-02-16T17:28:19.131Z level=INFO source=routes.go:1739 msg="vram-based default context" total_vram="5.3 GiB" default_num_ctx=4096

It hasnt done anything in a while whats going on?

That output is **exactly what you want** — `ollama serve` is a **server process**. Once it says:

- `Listening on 127.0.0.1:11434`
- and it's found your GPU (`Apple M2` / Metal)

…it will just **sit there idle** until something sends it a request. Nothing is "stuck".

What's happening in your log

- It generated an SSH key for Ollama's internal use (normal on first run).
- It started the API server on **127.0.0.1:11434**
- It detected your GPU and chose a default context length.
- Now it's waiting for requests.

✅ Next: verify it's actually responding

Open a **new Terminal tab/window** (leave `ollama serve` running in the first one) and run:

1) Check server is alive

```
curl http://127.0.0.1:11434/api/tags
```

You should get JSON back (probably with an empty `models` list right now).

2) Pull a model (you likely have none yet — it said "total blobs: 0")

```
ollama pull llama3.1
```

3) Quick run test

```
ollama run llama3.1 "Say hello in one sentence."
```

If that works, your FastAPI endpoint will work too.

If `ollama pull llama3.1` feels slow

First pull can take a while because it's downloading GBs. You'll see progress output. After that, it's fast.

One more detail: don't run `ollama serve` every time manually (optional)

On macOS you can keep it running in the background via `brew services` :

```
brew services start ollama
```

Stop later with:

```
brew services stop ollama
```

If you paste the output of `curl http://127.0.0.1:11434/api/tags` , I'll tell you whether you already have any models installed and what to do next.

(base) cayden@Caydens-MacBook-Air WorkoutAPI_WebData_Yr3 % curl http://127.0.0.1:11434/api/tags

{"models":[]}%
(base) cayden@Caydens-MacBook-Air WorkoutAPI_WebData_Yr3 % ollama pull llama3.1

pulling manifest
pulling 667b0c1932bc: 100%
[redacted] 4.9 GB
pulling 948af2743fc7: 100%
[redacted] 1.5 KB
pulling 0ba8f0e314b4: 100%
[redacted]

```
[redacted] 12 KB
pulling 56bb8bd477a5: 100%
[redacted] 96 B
pulling 455f34728c9b: 100%
[redacted] 487 B
verifying sha256 digest
writing manifest
success
(base) cayden@Caydens-MacBook-Air
WorkoutAPI_WebData_Yr3 % ollama run llama3.1 "Say
hello in one sentence."

Hello, how can I assist you today?

(base) cayden@Caydens-MacBook-Air
WorkoutAPI_WebData_Yr3 %
```

Perfect — Ollama is working end-to-end ✅

Now you're ready to wire your FastAPI endpoint to it.

1) Quick check: the model is installed

Run:

```bash
curl http://127.0.0.1:11434/api/tags
```

You should now see `llama3.1` listed.

---

2) Make your API use Ollama automatically

Install dependency

Bash

```bash
pip install httpx
```

Add env vars (optional but nice)

In your project `.env` :

```
OLLAMA_BASE_URL=http://127.0.0.1:11434
OLLAMA_MODEL=llama3.1
OLLAMA_TIMEOUT_SECONDS=90
```

Ensure `.env` loads at startup

At the top of your `main.py` (or wherever `app = FastAPI()` is created):

Python Run

```python
from dotenv import load_dotenv
load_dotenv()
```

---

3) Drop-in `llm_client.py` (Ollama)

Use this (same as before, but tuned slightly for llama3.1 + consistent output):

Python Run

```python
# app/ai/llm_client.py
from __future__ import annotations

import os
from typing import Any, Dict, Optional

import httpx


class LLMClient:
    def __init__(
        self,
        base_url: Optional[str] = None,
        model: Optional[str] = None,
        timeout_seconds: Optional[float] = None,
    ) -> None:
        self.base_url = (base_url or os.getenv("OLLAMA_BASE_URL", "http://
        self.model = model or os.getenv("OLLAMA_MODEL", "llama3.1")
        self.timeout = timeout_seconds or float(os.getenv("OLLAMA_TIMEOUT_
```

```
    def generate_weekly_summary(self, facts: Dict[str, Any], tone: str = "
        tone = (tone or "friendly").strip().lower()

        system_instructions = f"""
You are a fitness coach writing a weekly recap inside a workout tracker.

CRITICAL RULES:
- Use ONLY numbers/dates in the provided JSON. Do NOT invent workouts, set
- If a PR has "previous": null, treat it as a first recorded benchmark (do
- If "previous" exists, you may say it improved and compute the difference
- Do not assume units; say "weight" and "training volume" without units.

OUTPUT FORMAT (exactly):
Headline: <1 sentence>
Summary: <2 sentences max>

Highlights:
- <3–6 bullets>

Next week focus:
- <2–4 bullets>

Data notes:
- <1–3 bullets>
""".strip()

        user_content = f"""
Tone: {tone}

Weekly summary JSON:
{facts}

Write the weekly recap now.
""".strip()

        payload = {
            "model": self.model,
            "messages": [
                {"role": "system", "content": system_instructions},
                {"role": "user", "content": user_content},
            ],
            "stream": False,
            "options": {
                "temperature": 0.5,
                "num_predict": 450,  # caps output length to keep it tight
            },
        }

        url = f"{self.base_url}/api/chat"
        with httpx.Client(timeout=self.timeout) as client:
            r = client.post(url, json=payload)
            r.raise_for_status()
```

```
            data = r.json()

        content = (data.get("message") or {}).get("content")
        if not content:
            return f"[ollama] Unexpected response shape: {data}"
        return content.strip()
```

---

4) Add your “AI summary” endpoint

Example router:

Python Run

```python
# app/routers/ai_summaries.py
from datetime import date
from typing import Literal

from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session

from app.database import get_db
from app.ai.llm_client import LLMClient
from app.services.weekly_facts import build_weekly_facts  # <- adjust to y

router = APIRouter(prefix="/ai", tags=["AI Summaries"])

Tone = Literal["friendly", "direct", "hype", "clinical"]


@router.get("/users/{user_id}/summaries/weekly/ai")
def weekly_ai_summary(
    user_id: int,
    date_from: date = Query(...),
    date_to: date = Query(...),
    tone: Tone = Query("friendly"),
    num_facts: int = Query(10, ge=1, le=50),
    include_facts: bool = Query(False),
    db: Session = Depends(get_db),
):
    facts = build_weekly_facts(db, user_id, date_from, date_to, top_prs_co
    llm = LLMClient()
    summary = llm.generate_weekly_summary(facts=facts, tone=tone)

    out = {
        "user_id": user_id,
        "date_from": str(date_from),
        "date_to": str(date_to),
        "tone": tone,
        "summary": summary,
```

```
    }
    if include_facts:
        out["facts"] = facts
    return out
```

Register it:

Python Run

```python
app.include_router(ai_summaries_router)
```

---

5) Test the endpoint

Bash

```bash
curl -X 'GET' \
'http://127.0.0.1:8000/ai/users/2/summaries/weekly/ai?date_from=2026-02-09
-H 'accept: application/json'
```

One important note

Your FastAPI server and Ollama are now two processes:

- Ollama runs on `127.0.0.1:11434`
- Your API runs on `127.0.0.1:8000`

If you stop Ollama, your endpoint should return a clean error (we can add nicer handling if you want).

If you paste your current project structure (where `main.py` is + where `build_weekly_facts` lives), I'll adjust the imports so it compiles first time with your codebase.

(base) cayden@Caydens-MacBook-Air WorkoutAPI_WebData_Yr3 % curl http://127.0.0.1:11434/api/tags

```
{"models":[]}%
(base) cayden@Caydens-MacBook-Air
WorkoutAPI_WebData_Yr3 % ollama pull llama3.1

pulling manifest
pulling 667b0c1932bc: 100%
[redacted] 4.9 GB
pulling 948af2743fc7: 100%
[redacted] 1.5 KB
pulling 0ba8f0e314b4: 100%
[redacted] 12 KB
pulling 56bb8bd477a5: 100%
[redacted] 96 B
pulling 455f34728c9b: 100%
[redacted] 487 B
verifying sha256 digest
writing manifest
success
(base) cayden@Caydens-MacBook-Air
WorkoutAPI_WebData_Yr3 % ollama run llama3.1 "Say
hello in one sentence."

Hello, how can I assist you today?

(base) cayden@Caydens-MacBook-Air
WorkoutAPI_WebData_Yr3 %
```

this took forever to run

Yeah — that first “real” run *can* feel slow on an M2 Air, and there are two separate things going on here:

Why it felt like it took forever

1) You re-downloaded the model

Your `curl /api/tags` still returning `{"models":[]}` means Ollama wasn’t seeing any installed models at that moment. Then `ollama pull llama3.1` downloaded **4.9GB** again (that’s the “forever” part).

That usually happens when either:

- you were running `ollama serve` under a different user/environment than the CLI, **or**
- the server wasn’t the one you thought (rare), **or**
- the model store path differs (less common).

Your server log earlier showed models path:
`OLLAMA_MODELS:/Users/cayden/.ollama/models`
So it *should* persist.

2) First inference is slow (cold start)

Even after the model is present:

- First run loads weights into memory / GPU (Metal)
- Does initial compilation/warmup
- Subsequent runs are much faster

Fix the “models[]” issue first (so you don’t keep re-pulling)

A) Confirm where the CLI thinks models live

Run:

Bash

```
echo $OLLAMA_MODELS
ls -la ~/.ollama/models
```

You should see folders like `blobs` and `manifests`.

B) Confirm the server is using the same model directory

In the terminal running `ollama serve`, it prints:

`OLLAMA_MODELS:/Users/cayden/.ollama/models`

If your `echo $OLLAMA_MODELS` is empty, that's fine — default is `~/.ollama/models`. But if it's set to something else in one terminal, that would explain the mismatch.

C) After pulling, restart the server once

1. Stop the server (`Ctrl+C` in the serve window)
2. Start it again:

```bash
ollama serve
```

3. In a new terminal:

```bash
curl http://127.0.0.1:11434/api/tags
```

If it's fixed, you'll see `llama3.1` in the list and you will **not** have to re-pull again.

Make runs faster on your machine

1) Use a smaller model (recommended for M2 Air)

`llama3.1` in Ollama is often the larger variant — great, but heavier.

Try:

```bash
ollama pull llama3.1:8b
ollama run llama3.1:8b "Say hello in one sentence."
```

Or even faster:

```bash
ollama pull qwen2.5:7b
ollama run qwen2.5:7b "Say hello in one sentence."
```

These are *way* snappier for summaries.

2) Keep it warm

If the model unloads between calls, you'll get repeated cold starts. You can increase keep-alive:

Run server with:

```bash
OLLAMA_KEEP_ALIVE=30m ollama serve
```

(or set it in your shell / `.env` if you want)

3) Use shorter outputs for your endpoint

In the Ollama client options, keep `num_predict` capped (like 300–500) — you already have that.

What I'd do for your project (fast + reliable)

- Use `qwen2.5:7b` or `llama3.1:8b`
- Keep `OLLAMA_KEEP_ALIVE=30m`
- Summaries will feel near-instant after first call

Quick "tell me what's wrong" command

Paste the output of these (no secrets in there):

```bash
ls -la ~/.ollama/models
curl http://127.0.0.1:11434/api/tags
```

If the directory clearly contains manifests/blobs but `/api/tags` shows empty, I'll tell you exactly what's mismatched and how to fix it in one step.

{"models":[{"name":"llama3.1:latest","model":"llama3.1:latest","modified_at":"2026-02-16T17:35:18.814835011Z","size":4920753328,"digest":"46e0c10c039e019119339687c3c1757cc81b9da49709a3b3924863ba87ca666e","details":{"parent_model":"","format":"gguf","family":"llama","families":["llama"],"parameter_size":"8.0B","quantization_level":"Q4_K_M"}}]}%

I now have these models

Nice — that's sorted ✅

You've got `llama3.1:latest` **(8B, Q4_K_M)** installed and the server can see it now. The "took forever" is mostly:

- the **first download** (5GB)
- the **first load/warm start** of the model

From here, it should be much quicker, and we can make it *feel* fast for your API.

Make it faster for your endpoint (2 simple tweaks)

1) Keep the model in memory longer

Restart the server with a longer keep-alive so it doesn't unload between requests:

```bash
OLLAMA_KEEP_ALIVE=30m ollama serve
```

(Leave that terminal running.)

2) Use the exact model name in `.env`

Set:

```
OLLAMA_MODEL=llama3.1:latest
```

This avoids any “alias resolution” weirdness.

Update the client to be snappy + stable

In your `LLMClient` payload, add two options:

- `num_predict` (caps output)
- `temperature` lower for consistency

Python Run

```python
"options": {
    "temperature": 0.4,
    "num_predict": 350,
}
```

That helps a lot for “weekly recap” style.

Quick benchmark (you’ll *see* the improvement)

Run twice:

Bash

```bash
time ollama run llama3.1:latest "Say hello in one sentence."
time ollama run llama3.1:latest "Say hello in one sentence."
```

First run is often slower; second should be much faster if it stays warm.

Next step: wire into FastAPI

At this point your AI endpoint should work with the `llm_client.py` I gave, with only these changes:

- model default: `llama3.1:latest`
- cap output tokens (`num_predict`)
- optional keep-alive env

If you paste your current `uvicorn ...` command and where your `FastAPI()` app lives (e.g. `app/main.py`), I’ll give you the exact file paths + imports so it runs first try.

Can you give me a new llm_client using these new changes

Python Run

```python
# app/ai/llm_client.py
from __future__ import annotations

import os
from typing import Any, Dict, Optional

import httpx


class LLMClient:
    """
    Local LLM client using Ollama (free, runs on your machine).

    Env vars (optional):
      - OLLAMA_BASE_URL (default: http://127.0.0.1:11434)
      - OLLAMA_MODEL (default: llama3.1:latest)
      - OLLAMA_TIMEOUT_SECONDS (default: 90)
      - OLLAMA_TEMPERATURE (default: 0.4)
      - OLLAMA_NUM_PREDICT (default: 350)  # caps output length
    """

    def __init__(
        self,
        base_url: Optional[str] = None,
        model: Optional[str] = None,
        timeout_seconds: Optional[float] = None,
        temperature: Optional[float] = None,
        num_predict: Optional[int] = None,
    ) -> None:
        self.base_url = (base_url or os.getenv("OLLAMA_BASE_URL", "http://
        self.model = model or os.getenv("OLLAMA_MODEL", "llama3.1:latest")
        self.timeout = timeout_seconds or float(os.getenv("OLLAMA_TIMEOUT_

        # Generation options
        self.temperature = temperature if temperature is not None else flo
        self.num_predict = num_predict if num_predict is not None else int

    def generate_weekly_summary(self, facts: Dict[str, Any], tone: str = "
        tone = (tone or "friendly").strip().lower()
```

```
        system_instructions = f"""
You are a fitness coach writing a weekly recap inside a workout tracker.

CRITICAL RULES:
- Use ONLY numbers and dates in the provided JSON. Do NOT invent workouts,
- If a PR has "previous": null, treat it as a first recorded benchmark (do
- If "previous" exists, you may say it improved and compute/mention the di
- Do not assume any units; say "weight" and "training volume" without unit
- Only include a safety note if pain/injury is explicitly mentioned in the

OUTPUT FORMAT (exactly):
Headline: <1 sentence>
Summary: <2 sentences max>

Highlights:
- <3–6 bullets>

Next week focus:
- <2–4 bullets>

Data notes:
- <1–3 bullets>
""".strip()

        user_content = f"""
Tone: {tone}

Weekly summary JSON:
{facts}

Write the weekly recap now.
""".strip()

        payload = {
            "model": self.model,
            "messages": [
                {"role": "system", "content": system_instructions},
                {"role": "user", "content": user_content},
            ],
            "stream": False,
            "options": {
                "temperature": self.temperature,
                "num_predict": self.num_predict,
            },
        }

        url = f"{self.base_url}/api/chat"

        try:
            with httpx.Client(timeout=self.timeout) as client:
                r = client.post(url, json=payload)
```

```
            r.raise_for_status()
            data = r.json()

        content = (data.get("message") or {}).get("content")
        if not content:
            return f"[ollama] Unexpected response shape: {data}"
        return content.strip()

    except httpx.ConnectError:
        return (
            "Could not connect to Ollama. Make sure it's running:\n"
            "  OLLAMA_KEEP_ALIVE=30m ollama serve\n"
            f"and that OLLAMA_BASE_URL is correct (currently {self.bas
        )
    except httpx.HTTPStatusError as e:
        return f"Ollama error ({e.response.status_code}): {e.response.
    except Exception as e:
        return f"Unexpected error calling Ollama: {e}"
```